Technical Report: Dog Adoption Management System

Bazon Bogdan
Crismariu Denis
Meraru Ioan-Lucian
Vezeteu Andrei

May 21, 2026

Contents

Chapter 1

Introduction

In the contemporary digital era, the intersection of animal welfare and technology has given rise to innovative solutions aimed at addressing long-standing challenges in the pet adoption process. This technical report details the development and architecture of a sophisticated Dog Adoption Management System, a platform designed to streamline the journey from shelter to forever home.

The primary objective of this project is to create a robust, scalable, and user-centric ecosystem that empowers both animal shelters and potential adopters. By leveraging modern cloud-native technologies, microservices architecture, and automated CI/CD pipelines, we aim to provide a seamless experience that reduces the friction inherent in traditional adoption workflows.

Pet adoption is not merely a transactional process; it is a complex emotional and administrative journey. Shelters often struggle with fragmented data, manual record-keeping, and limited reach. Adopters, on the other hand, frequently face opaque application processes, lack of real-time updates, and difficulty in finding the right companion. This project addresses these pain points by centralizing information, automating notifications, and providing an intuitive interface for all stakeholders.

The system is architected as a distributed network of specialized services, including User Management, Pet Management, Adoption Tracking, and Review Services. Each component is designed with a "Clean Architecture" approach, ensuring a high degree of maintainability, testability, and independence from external frameworks. Persistence is handled through a combination of relational databases for structured data and NoSQL stores for flexible, document-based application storage.

This report will explore the theoretical underpinnings of the project through a detailed case study, analyze the landscape of existing solutions, and provide a deep dive into the technical implementation. We will also examine the business implications of the platform and provide comprehensive documentation of its APIs and workflows.

Chapter 2

Case Study: Pet Adoption Challenges and Opportunities

2.1 The Landscape of Pet Abandonment

Pet abandonment remains a global crisis with profound ethical, social, and economic implications. Every year, millions of domestic animals, particularly dogs, end up in shelters or on the streets. According to global animal welfare organizations, the reasons for abandonment range from socio-economic factors to a lack of understanding of the responsibilities associated with pet ownership. In many cases, behavioral issues, life transitions (such as moving or the birth of a child), and financial constraints drive owners to surrender their companions.

The impact on the animals is devastating. Dogs, being highly social and intelligent creatures, experience intense stress and anxiety when displaced from a familiar environment. Shelters, while providing a safe haven, are often overcrowded and under-resourced, leading to a "warehousing" effect where animals may spend months or even years in confined spaces with limited human interaction.

2.2 The Global Crisis of Pet Overpopulation

Pet abandonment is a subset of the larger issue of pet overpopulation. According to organizations like the World Health Organization and various international animal welfare groups, there are estimated to be over 200 million stray dogs worldwide. In many developing nations, this poses a significant public health risk (e.g., rabies transmission), while in more developed nations, the burden falls primarily on the shelter infrastructure.

Statistics indicate that in the United States alone, approximately 6.3 million companion animals enter shelters every year. Of these, roughly 3.1 million are dogs. While adoption rates have been steadily increasing due to public awareness campaigns, approximately 390,000 shelter dogs are still euthanized annually. This is often not due to behavioral or health issues, but simply due to a lack of space and the administrative inability to find homes for them before new intakes arrive.

2.3 The Economic Cost of Abandonment

The economic impact of pet abandonment is often overlooked. Municipalities spend billions of dollars collectively on animal control, shelter operations, and public health measures related to strays. A single "intake to adoption" cycle for a dog can cost a shelter anywhere from $300 to $800, covering food, medical care, spaying/neutering, and marketing.

For many shelters, the "cost per day" of housing an animal is the most critical metric. When administrative inefficiencies (like the ones this project aims to solve) delay an adoption by even 5 days, the cumulative cost across hundreds of animals can be the difference between a shelter staying solvent or being forced to reduce services.

2.4 The Psychological Dimension: Shelter Stress

Modern veterinary science has extensively documented "shelter stress." Dogs in a shelter environment often exhibit repetitive behaviors, chronic anxiety, and suppressed immune systems. This creates a "vicious cycle": a stressed dog is less likely to show well to a potential adopter, which leads to a longer stay in the shelter, which in turn increases the stress and worsens the behavior.

A digital system that fast-tracks the application and matching process directly contributes to the physical and mental well-being of the animals by reducing their "length of stay" (LOS). Optimizing LOS is the gold standard for modern shelter management, and it is a primary goal of the Dog Adoption Management System.

2.5 The Administrative Crisis in Shelters

Most animal shelters operate as non-profit organizations or municipal departments with very tight budgets. The administrative burden is immense. Staff and volunteers must manage:

- **Intake and Medical Records:** Documenting each animal's history, health status, vaccinations, and behavioral assessments.
- **Application Management:** Processing hundreds of adoption applications, conducting background checks, and scheduling interviews.

- **Inventory Tracking:** Keeping an up-to-date record of available pets, pending adoptions, and successful placements.
- **Public Outreach:** Marketing animals through various channels to find potential adopters.

The reliance on manual or fragmented systems leads to inefficiencies. For example, a dog might be listed as "Available" on a website long after an application has been approved, leading to frustration for potential adopters and redundant work for staff. Data silos also prevent effective communication between different departments or branches of a shelter network.

2.6 The Adopter's Journey: Friction and Barriers

From the perspective of a potential adopter, the process can be equally daunting. A typical journey involves:

1. **Search:** Browsing multiple websites and social media pages to find a compatible dog.
2. **Inquiry:** Reaching out to shelters, often receiving delayed or no responses due to staff shortages.
3. **Application:** Filling out lengthy, often paper-based forms that ask repetitive questions.
4. **Waiting:** Waiting for weeks without knowing the status of their application.

High friction in the adoption process often drives well-meaning individuals toward pet stores or unregulated breeders, inadvertently supporting unethical breeding practices (such as puppy mills). A streamlined, transparent digital process is essential to make adoption the first and easiest choice for anyone looking for a companion.

2.7 Opportunities for Technological Intervention

The challenges identified above present a significant opportunity for a modern, integrated platform. By digitizing and automating key parts of the workflow, we can achieve:

- **Real-time Data Synchronization:** Ensuring that pet availability is accurate across all platforms.
- **Automated Workflow Tracking:** Moving applications through stages (Submitted, Review, Interview, Approved) with automatic notifications to the applicant.

- **Scalable Communication:** Using event-driven systems to trigger emails or SMS updates, reducing the manual workload on staff.
- **Data-Driven Insights:** Collecting analytics on adoption trends to help shelters allocate resources more effectively.

This case study demonstrates that the problem is not a lack of interest in adoption, but rather a structural inefficiency in the systems that facilitate it. The Dog Adoption Management System is designed to bridge this gap, transforming a fragmented manual process into a cohesive, automated, and human-centric experience.

Chapter 3

Technical Details of Existing Solutions

To understand the value proposition of the Dog Adoption Management System, it is crucial to analyze the landscape of currently available tools. Existing solutions generally fall into three categories: Legacy Shelter Management Software, Aggregator Marketplaces, and Generic CRM/Internal tools.

3.1 Legacy Shelter Management Software

These are specialized applications designed specifically for shelters. Examples include "ShelterBuddy" and "PetPoint".

3.1.1 Architecture and Technology

Historically, these systems were built as monolithic desktop applications or early web-based platforms. Many still rely on on-premise servers or outdated cloud models. Their architectures often suffer from:

- **High Coupling:** A change in the billing module can impact the pet medical records module. This architectural debt makes it nearly impossible to implement rapid feature updates or pivot to new technologies like mobile-first reactive interfaces.
- **Data Inaccessibility:** Many legacy providers charge additional fees for "Data Export" or "API Access," effectively locking shelters into their ecosystem.
- **Slow Performance:** Large, complex databases that are not optimized for modern high-concurrency needs.

3.1.2 Marketing and Business Model

The business model for legacy software is often based on high upfront licensing costs and per-user seat pricing. This model is profitable for the vendor but creates a financial barrier for organizations that rely on large numbers of part-time volunteers. Their marketing focuses on compliance and insurance requirements rather than the speed of adoption or the user experience of the adopter.

3.2 Aggregator Marketplaces

Platforms like Petfinder and Adopt-a-Pet are the most visible solutions in the market.

3.2.1 Architecture and Technology

These are modern, web-scale platforms optimized for search and discovery. They use sophisticated caching, search indexing (like Elasticsearch), and CDNs to serve millions of images and search queries. Their architecture is designed to handle massive amounts of read-only traffic. However, their "Backend-as-a-Service" (BaaS) for shelters is often an afterthought, lacking the deep workflow management needed for daily operations.

3.2.2 Marketing and Consumer Dominance

These platforms have a virtual monopoly on consumer search traffic for pets. Any new solution must either integrate with these marketplaces via APIs or provide a superior enough "Direct-to-Consumer" experience that shelters can build their own brand presence. Currently, marketplaces own the adopter relationship, leaving the shelter as a mere "fulfillment center."

3.3 Generic CRM and Internal Tools

Some organizations use general-purpose tools like Salesforce or Trello to manage adoptions.

3.3.1 Architecture and Technology

These tools offer massive flexibility through low-code or no-code platforms. Architecture is typically multi-tenant SaaS. While they are highly secure and reliable, they require a "custom developer" to map pet welfare requirements to generic "Sales Pipeline" fields. For example, a "Lead" becomes an "Adopter," and a "Deal" becomes an "Adoption."

3.3.2 Marketing and Adoption

They are marketed as versatile productivity tools. While they can be adapted for adoption workflows, the customization process is expensive and often results in a "clunky" user experience that doesn't feel native to the animal rescue mission.

3.4 Niche SaaS Startups

Recently, a few "cloud-native" startups have emerged (e.g., Shelterluv).

3.4.1 Architecture and Technology

These are built on modern web stacks (React, Node.js/Python, AWS/GCP). They offer better UX and simple REST APIs. However, many still struggle with "Horizontal Scaling" or are built on monolithic cloud architectures that make them vulnerable to downtime when traffic spikes during national adoption events.

3.5 Gap Analysis: Why a New Solution is Needed

The analysis of existing solutions reveals several critical gaps:

1. **Integration Deficit:** There is a lack of "Management-as-a-Service" platforms that offer both internal administrative control and a high-quality, modern adopter interface.
2. **Technological Stagnation:** Legacy tools are not utilizing the full potential of cloud-native architectures (microservices, event-driven scaling, and automated DevOps).
3. **Cost Inaccessibility:** High licensing fees prevent smaller shelters from digitizing their operations.
4. **UX Friction:** Most tools prioritize the database structure over the human experience (both for shelter staff and for adopters).

The Dog Adoption Management System addressed in this report is designed to fill these gaps by offering a microservices-based, scalable architecture that is both powerful enough for large shelters and intuitive enough for small rescues, with a focus on real-time integration and user-centric design.

Chapter 4

Overview of Technologies and Motivation

The Dog Adoption Management System is built using a modern, cloud-native stack designed for high availability, scalability, and maintainability. The selection of these technologies was driven by the need to handle complex business logic, varying traffic loads, and the requirement for a professional, responsive user experience.

4.1 Architectural Paradigm: Microservices and Clean Architecture

The project adopts a **Microservices Architecture**, which allows each functional area (User Management, Pet Management, Adoption Tracking, etc.) to be developed, deployed, and scaled independently. This approach prevents the "monolithic bottleneck" where a failure in one module can crash the entire system.

Within each microservice, we implement **Clean Architecture** (also known as Onion Architecture). This paradigm organizes code into concentric layers:

- **Domain Layer:** Contains the core business entities and logic, completely independent of any external frameworks.
- **Application Layer:** Implements use cases and coordinates the flow of data using the CQRS (Command Query Responsibility Segregation) pattern with the **MediatR** library.
- **Infrastructure Layer:** Handles technical concerns such as database persistence and external API integrations.
- **API/Presentation Layer:** Exposes the functionality through RESTful endpoints.

Motivation: Clean Architecture ensures that the core business rules are easy to test and resistant to changes in external technologies (like switching databases or frameworks).

4.2 Backend Development: .NET 10 (Web API)

The backend services are built using **.NET 10**, the latest long-term support release from Microsoft. .NET was chosen for its high performance, cross-platform capabilities, and robust ecosystem for building enterprise-grade APIs.

Key features used include:

- **Minimal APIs:** For lightweight, high-performance routing.
- **Entity Framework Core:** For Object-Relational Mapping (ORM) with PostgreSQL.
- **Dependency Injection:** Built-in support for managing service lifetimes and promoting loose coupling.

Motivation: .NET 10 provides industry-leading throughput and a massive library of NuGet packages that accelerate the development of complex features like JWT authentication and CosmosDB integration.

4.3 Frontend Development: React and Tailwind CSS

The user interface is a Single Page Application (SPA) built with **React**, the most popular library for building reactive web components.

- **Vite:** Used as the build tool for extremely fast development cycles and optimized production bundles.
- **Tailwind CSS:** A utility-first CSS framework that allows for rapid styling and a consistent "amber-themed" design language across the entire platform.
- **React Context API:** Used for global state management, such as authentication status and user profiles.

Motivation: React's component-based model allows us to reuse UI elements (like the dog adoption cards and application modals), while Tailwind ensures the application is fully responsive for mobile users.

4.4 Data Persistence Strategy

We utilize a polyglot persistence strategy, choosing the best database for each specific use case:

4.4.1 Relational: PostgreSQL

Used for services with highly structured data and complex relationships, such as User Management and Review Services. PostgreSQL provides strong ACID compliance and advanced querying capabilities.

4.4.2 NoSQL: Azure CosmosDB

Used for the Adoption Manager service. Since adoption applications can vary in structure and require high-speed, globally distributed access, a document-based store is ideal. *Motivation:* Polyglot persistence prevents the system from being forced into a "one size fits all" database model, optimizing both performance and developer productivity.

4.5 Cloud-Native Infrastructure and DevOps

The deployment and orchestration of the system are entirely automated:

- **Docker:** All services are containerized to ensure "build once, run anywhere" consistency between local development and production.
- **Azure Kubernetes Service (AKS):** Orchestrates the containers, handling auto-scaling, self-healing, and internal load balancing.
- **Nginx Ingress Controller:** Acts as the unified gateway, routing traffic from the public IP to the appropriate microservices based on URL paths.
- **GitHub Actions (CI/CD):** Automates the build and deployment process. We use **OpenID Connect (OIDC)** for secure, password-less authentication between GitHub and Azure.

Motivation: This "Infra-as-Code" approach reduces human error and allows the project to be updated multiple times a day with zero downtime.

4.6 Observability and Security

- **Azure Application Insights:** Provides real-time telemetry, allowing us to track "Pet Views" vs. "Applications Submitted" and monitor for system errors.
- **JWT (JSON Web Tokens):** Ensures that all API calls are securely authenticated and that user data is protected.

Motivation: High-quality telemetry is essential for understanding user behavior, while standard-compliant security is non-negotiable for a platform handling personal data.

Chapter 5

Business Canvas

Chapter 6

Architecture Diagram

Chapter 7

Use Case Diagram

Chapter 8

Functionality, Flow, and API Documentation